

Fundamentals of Linear Algebra for Machine Learning and Deep Learning

Nylan Richard

March 16, 2025

Preface

Linear algebra is the mathematics of vectors and matrices, and it forms the foundation of many algorithms in machine learning (ML) and deep learning (DL). This book aims to provide a comprehensive introduction to linear algebra with a focus on its applications in ML and DL. We start from basic concepts accessible to beginners and progress to advanced topics that underpin modern AI techniques. Each chapter includes theoretical explanations, examples, and exercises with detailed step-by-step solutions to reinforce understanding. Readers are encouraged to work through the exercises, as hands-on practice is essential for mastering the material. We assume only a basic background in algebra and calculus.

Contents

Preface	i
1 Introduction	1
2 Fundamental Concepts	3
2.1 Scalars, Vectors, and Matrices	3
2.2 Vector Operations	3
2.3 Matrix Operations	5
2.4 Systems of Linear Equations and Gaussian Elimination	6
3 Intermediate Concepts	9
3.1 Determinants and Matrix Inverses	9
3.2 Transpose of a Matrix	10
3.3 Vector Spaces and Subspaces	10
3.4 Linear Independence, Basis, and Rank	10
3.5 Linear Transformations	11
4 Advanced Concepts	14
4.1 Eigenvalues and Eigenvectors	14
4.2 Eigendecomposition and Diagonalization	15
4.3 LU and QR Decompositions	15
4.4 Singular Value Decomposition (SVD)	16
4.5 Orthogonal Projections	17
5 Advanced Topics for ML/DL	19
5.1 Principal Component Analysis (PCA)	19
5.2 SVD and Data Compression	20
5.3 Eigenvalues in Dimensionality Reduction	21
5.4 Optimization: Gradient Descent and the Hessian	21

Chapter 1

Introduction

Linear algebra is a branch of mathematics concerned with vectors, matrices, and linear transformations. It is a foundational tool in many fields, especially in machine learning (ML) and deep learning (DL), where data and model parameters are often represented as vectors and matrices. In ML and DL, we routinely represent datasets as matrices (e.g., a table of feature values) and parameters of models as vectors or higher-dimensional tensors. Operations such as computing predictions, losses, and gradients fundamentally rely on linear algebraic computations like vector dot-products and matrix multiplications.

Importance of Linear Algebra in ML/DL

Linear algebra provides the language for expressing and solving many problems in ML/DL. For instance, training a linear regression model involves solving linear equations to find the best-fit line to data. Neural networks, which power modern deep learning, use weight matrices to transform input vectors through layers. Concepts like eigenvalues and eigenvectors form the basis of principal component analysis (PCA), a common dimensionality reduction technique. Optimizing ML models often requires understanding gradients and Hessians, which are expressed using vectors and matrices. Without a firm grasp of linear algebra, it is difficult to understand how and why many ML/DL algorithms work.

Beyond computations, linear algebra also offers deep insight into why algorithms behave a certain way. Many ML algorithms can be interpreted in terms of geometric transformations in vector spaces. For example, understanding how a dataset's variance is oriented in space (via PCA) can explain which features are most significant. Understanding linear transformations helps in grasping concepts such as convolution in neural networks or the effect of orthogonal weight initialization.

Scope and Structure of this Book

This book provides a comprehensive guide to linear algebra with a focus on applications in ML and DL. We start with fundamental concepts (scalars, vectors, matrices, and basic operations) in Chapter 2, ensuring that even readers with minimal background can follow. Chapter 3 introduces intermediate topics like determinants, inverses, and vector spaces, which build the groundwork for more advanced ideas. Chapter 4 covers advanced linear algebra concepts such as eigenvalues, eigenvectors, and matrix decompositions (LU, QR, SVD) that are crucial in understanding advanced ML techniques. In Chapter 5, we

delve into advanced topics at the intersection of linear algebra and ML/DL, including principal component analysis (PCA), the use of singular value decomposition in data compression, the role of eigenvalues in dimensionality reduction, and an overview of optimization techniques like gradient descent in the context of linear algebra.

Each chapter provides examples and exercises that relate the math to real-world ML/DL applications. At the end of each chapter, a set of exercises is provided along with step-by-step solutions to solidify understanding. Readers are encouraged to attempt the exercises on their own before reading the solutions. All mathematical notation is presented using standard LaTeX formatting for clarity, and figures are generated with TikZ to illustrate key concepts visually.

We assume the reader has basic knowledge of algebra and calculus, but we start from the basics of linear algebra. By the end of this book, readers should be comfortable with the linear algebraic concepts and techniques used in modern machine learning and deep learning. We hope that this material not only teaches computational procedures but also provides geometric intuition that will help in the design and understanding of ML/DL models.

For further reading on these topics, standard texts such as Strang's linear algebra book and the Deep Learning textbook by Goodfellow et al. [1, 2] have been influential sources.

Chapter 2

Fundamental Concepts

2.1 Scalars, Vectors, and Matrices

In linear algebra, the basic elements are scalars, vectors, and matrices. A **scalar** is a single number. Scalars in this book will typically be real numbers (denoted $\mathbb{R}$), although linear algebra can be extended to complex numbers or other fields. We often denote scalars with lowercase letters (e.g., $a, b, c \in \mathbb{R}$).

A **vector** is an ordered list of numbers. Vectors can be thought of as points in a geometric space or as arrows with direction and magnitude. For example, a vector in $\mathbb{R}^3$ (3-dimensional space) can be written as:

$$\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix},$$

where v_1, v_2, v_3 are the components of the vector. We typically denote vectors with bold lowercase letters (e.g., $\mathbf{v}, \mathbf{w}$) or sometimes with an arrow overhead (e.g., $\vec{v}$). The set of all n -dimensional vectors of real numbers is denoted $\mathbb{R}^n$.

A **matrix** is a rectangular array of numbers, arranged in rows and columns. An example of a 2×3 matrix (2 rows, 3 columns) is:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix},$$

where a_{ij} denotes the entry in the i -th row and j -th column. We often denote matrices with bold capital letters (e.g., $\mathbf{A}, \mathbf{B}$). The set of all $m \times n$ matrices with real entries is denoted $\mathbb{R}^{m \times n}$. Matrices can represent data (e.g., a dataset with m examples each having n features can be seen as an $m \times n$ matrix) or linear transformations between vector spaces.

2.2 Vector Operations

Vectors can be added together or scaled by numbers (scalars), as long as they have the same dimension. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$. The sum $\mathbf{u} + \mathbf{v}$ is a new vector obtained by

adding corresponding components:

$$\mathbf{u} + \mathbf{v} = \begin{pmatrix} u_1 + v_1 \\ u_2 + v_2 \\ \vdots \\ u_n + v_n \end{pmatrix}.$$

Scalar multiplication produces a new vector by multiplying each component of $\mathbf{v}$ by the scalar α :

$$\alpha \mathbf{v} = \begin{pmatrix} \alpha v_1 \\ \alpha v_2 \\ \vdots \\ \alpha v_n \end{pmatrix}.$$

These operations satisfy the usual arithmetic properties: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$ (commutativity), $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition), and $\alpha(\mathbf{u} + \mathbf{v}) = \alpha\mathbf{u} + \alpha\mathbf{v}$ (distributivity). The zero vector $\mathbf{0} = (0, 0, \dots, 0)$ acts as an additive identity: $\mathbf{v} + \mathbf{0} = \mathbf{v}$.

One way to visualize vector addition is the parallelogram rule. If we draw vectors $\mathbf{u}$ and $\mathbf{v}$ as arrows from the origin, $\mathbf{u} + \mathbf{v}$ corresponds to the diagonal of the parallelogram formed by $\mathbf{u}$ and $\mathbf{v}$:

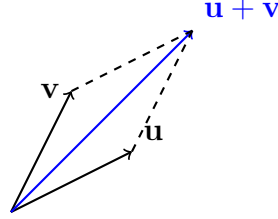


Figure 2.1: Vector addition $\mathbf{u} + \mathbf{v}$ visualized as the diagonal of a parallelogram spanned by $\mathbf{u}$ and $\mathbf{v}$.

Another important operation is the **dot product** (also called the inner product) between two vectors of the same length. For $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, the dot product is defined as:

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \dots + u_n v_n = \sum_{i=1}^n u_i v_i.$$

The dot product is a scalar. Geometrically, if θ is the angle between $\mathbf{u}$ and $\mathbf{v}$ when they are placed tail-to-tail, then:

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta,$$

where $\|\mathbf{u}\|$ denotes the length (or norm) of $\mathbf{u}$. This definition reveals that the dot product is positive if the angle between vectors is acute ($\cos \theta > 0$), negative if the angle is obtuse, and zero if the vectors are orthogonal (perpendicular).

The **norm** of a vector $\mathbf{v}$, denoted $\|\mathbf{v}\|$, measures its length (magnitude). The most common norm is the Euclidean (or L^2) norm:

$$\|\mathbf{v}\|_2 = \sqrt{\mathbf{v} \cdot \mathbf{v}} = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}.$$

For example, the length of $\mathbf{v} = (3, 4)$ in $\mathbb{R}^2$ is $\sqrt{3^2 + 4^2} = 5$. Norms are used in ML to quantify sizes of vectors (e.g., the size of weight vectors in regularization).

Another useful measure is the L^1 norm (also called Manhattan norm) $\|\mathbf{v}\|_1 = |v_1| + |v_2| + \dots + |v_n|$, which sums absolute values of components. There is also the max norm (or L^∞ norm) defined as $\|\mathbf{v}\|_\infty = \max_i |v_i|$. Different norms appear in regularization and loss functions in ML (for instance, L^2 norm is used in Ridge regression, and L^1 in Lasso for inducing sparsity).

2.3 Matrix Operations

Matrices, like vectors, can be added or subtracted if they have the same dimensions (same number of rows and columns). If $\mathbf{A}$ and $\mathbf{B}$ are both $m \times n$ matrices, their sum $\mathbf{A} + \mathbf{B}$ is the $m \times n$ matrix with entries added componentwise:

$$(\mathbf{A} + \mathbf{B})_{ij} = A_{ij} + B_{ij}.$$

Scalar multiplication extends similarly: for scalar α , $(\alpha\mathbf{A})_{ij} = \alpha A_{ij}$. These operations satisfy similar properties (commutativity, associativity, distributivity) as their vector counterparts.

A key operation for matrices is **matrix multiplication**. If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{B}$ is an $n \times p$ matrix, then the product $\mathbf{C} = \mathbf{AB}$ is defined and will be an $m \times p$ matrix. The entry of $\mathbf{C}$ at row i and column j is given by:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

Each entry C_{ij} is the dot product of the i -th row of $\mathbf{A}$ with the j -th column of $\mathbf{B}$. For example,

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} a & b \\ c & d \\ e & f \end{pmatrix} = \begin{pmatrix} 1 \cdot a + 2 \cdot c + 3 \cdot e & 1 \cdot b + 2 \cdot d + 3 \cdot f \\ 4 \cdot a + 5 \cdot c + 6 \cdot e & 4 \cdot b + 5 \cdot d + 6 \cdot f \end{pmatrix}.$$

Matrix multiplication is not commutative in general; $\mathbf{AB}$ is generally not equal to $\mathbf{BA}$, and one or both might not even be defined if dimensions do not align. However, matrix multiplication is associative: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$ (when the dimensions are compatible).

Matrices can act as linear transformations. If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{x} \in \mathbb{R}^n$ is a vector, then $\mathbf{Ax} \in \mathbb{R}^m$ is a new vector. This product can be viewed as applying a transformation $\mathbf{A}$ to the vector $\mathbf{x}$. For example, in a neural network, an input vector $\mathbf{x}$ might be multiplied by a weight matrix $\mathbf{W}$ to produce an output $\mathbf{Wx}$ for the next layer. Understanding matrix multiplication is crucial for understanding how data moves through layers in deep learning.

Two special matrices deserve mention:

- The **identity matrix** I_n is the $n \times n$ matrix with 1's on the diagonal and 0's elsewhere. It acts as a multiplicative identity for matrices, meaning $I_n \mathbf{A} = \mathbf{A} I_n = \mathbf{A}$ for any $n \times n$ matrix $\mathbf{A}$.
- The **zero matrix** $\mathbf{0}$ has all entries zero (of whatever dimension is contextually appropriate). It acts as an additive identity: $\mathbf{A} + \mathbf{0} = \mathbf{A}$.

2.4 Systems of Linear Equations and Gaussian Elimination

A system of linear equations is a collection of equations of the form:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m, \end{aligned}$$

where $x_1, x_2, \dots, x_n$ are unknown variables and the coefficients a_{ij} and constants b_i are known scalars. Such a system can be compactly written in matrix form as $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the matrix of coefficients (a_{ij}) , $\mathbf{x} \in \mathbb{R}^n$ is the column vector of unknowns $(x_1, \dots, x_n)^T$, and $\mathbf{b} \in \mathbb{R}^m$ is the column vector of constants $(b_1, \dots, b_m)^T$.

For example, the system:

$$\begin{aligned} 2x + 3y &= 5, \\ x - y &= 0, \end{aligned}$$

can be written as

$$\begin{pmatrix} 2 & 3 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 5 \\ 0 \end{pmatrix}.$$

To solve such systems, a common method is **Gaussian elimination**. The idea is to use row operations to reduce the matrix $\mathbf{A}$ to an upper triangular form (or even to reduced row-echelon form), making the system easier to solve by back-substitution. The basic row operations are:

1. Swapping two rows.
2. Multiplying a row by a nonzero scalar.
3. Adding a scalar multiple of one row to another row.

These operations do not change the solution set of the system. Using these, one can solve the system step by step.

Example (Gaussian elimination): Solve the system:

$$\begin{aligned} 2x + 3y &= 8, \\ 4x + 1y &= 7. \end{aligned}$$

Solution: Write the augmented matrix and perform elimination:

$$\left[\begin{array}{cc|c} 2 & 3 & 8 \\ 4 & 1 & 7 \end{array} \right].$$

First, use the first equation to eliminate x from the second. Multiply the first row by 2 and subtract from the second row:

$$\text{Row2} \leftarrow \text{Row2} - 2 \times \text{Row1} : \quad \left[\begin{array}{cc|c} 2 & 3 & 8 \\ 0 & -5 & -9 \end{array} \right].$$

Now the second equation is $-5y = -9$, so $y = \frac{-9}{-5} = \frac{9}{5} = 1.8$. Substitute y back into the first equation $2x + 3(1.8) = 8$ to get $2x + 5.4 = 8$, so $2x = 2.6$ and $x = 1.3$. Thus the solution is $(x, y) = (1.3, 1.8)$.

Gaussian elimination is the basis for more advanced algorithms like finding matrix inverses and computing determinants. In computational contexts, it is closely related to the LU decomposition which we will discuss in a later chapter.

Exercises

1. **Vector operations practice:** Given $\mathbf{a} = (1, 2, 3)$ and $\mathbf{b} = (4, -1, 2)$ in $\mathbb{R}^3$:

- (a) Compute $\mathbf{a} + \mathbf{b}$ and $2\mathbf{a} - 3\mathbf{b}$.
- (b) Compute the dot product $\mathbf{a} \cdot \mathbf{b}$.
- (c) Compute the Euclidean norm $\|\mathbf{a}\|$.

Solution:

- (a) $\mathbf{a} + \mathbf{b} = (1 + 4, 2 + (-1), 3 + 2) = (5, 1, 5)$. Also, $2\mathbf{a} - 3\mathbf{b} = 2(1, 2, 3) - 3(4, -1, 2) = (2, 4, 6) - (12, -3, 6) = (-10, 7, 0)$.
- (b) $\mathbf{a} \cdot \mathbf{b} = 1 \cdot 4 + 2 \cdot (-1) + 3 \cdot 2 = 4 - 2 + 6 = 8$.
- (c) $\|\mathbf{a}\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{1 + 4 + 9} = \sqrt{14} \approx 3.74$.

2. **Matrix multiplication and identity:** Let $\mathbf{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ (the 2×2 identity)

and $\mathbf{M} = \begin{pmatrix} 2 & -1 \\ 3 & 4 \end{pmatrix}$. Compute $\mathbf{I}_2\mathbf{M}$ and $\mathbf{M}\mathbf{I}_2$ and verify that the result equals

$\mathbf{M}$. Then explicitly compute $\mathbf{M}^2 = \mathbf{M}\mathbf{M}$. **Solution:** $\mathbf{I}_2\mathbf{M} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & -1 \\ 3 & 4 \end{pmatrix} =$

$\begin{pmatrix} 1 * 2 + 0 * 3 & 1 * (-1) + 0 * 4 \\ 0 * 2 + 1 * 3 & 0 * (-1) + 1 * 4 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ 3 & 4 \end{pmatrix} = \mathbf{M}$. Similarly, $\mathbf{M}\mathbf{I}_2 = \mathbf{M}$. For $\mathbf{M}^2$:

$$\mathbf{M}^2 = \begin{pmatrix} 2 & -1 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 2 & -1 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 2 * 2 + (-1) * 3 & 2 * (-1) + (-1) * 4 \\ 3 * 2 + 4 * 3 & 3 * (-1) + 4 * 4 \end{pmatrix} = \begin{pmatrix} 4 - 3 & -2 - 4 \\ 6 + 12 & -3 + 16 \end{pmatrix} = \begin{pmatrix} 1 & -6 \\ 18 & 13 \end{pmatrix}$$

3. **Solving linear equations:** Solve the following system of equations using Gaussian elimination:

$$\begin{aligned} x + 2y - z &= 1, \\ 2x + y + 3z &= 14, \\ -x + 2y + 2z &= 7. \end{aligned}$$

Solution: Form the augmented matrix and reduce:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 1 \\ 2 & 1 & 3 & 14 \\ -1 & 2 & 2 & 7 \end{array} \right].$$

Use Row1 to eliminate x from Row2 and Row3. $\text{Row2} \leftarrow \text{Row2} - 2 \times \text{Row1}$:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 1 \\ 0 & -3 & 5 & 12 \\ -1 & 2 & 2 & 7 \end{array} \right].$$

Row3 $\leftarrow$ Row3 + Row1:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 1 \\ 0 & -3 & 5 & 12 \\ 0 & 4 & 1 & 8 \end{array} \right].$$

Now use the new Row2 to eliminate y from Row3. First, fix Row2 by multiplying by $-1/3$ to get a leading 1 for y :

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 1 \\ 0 & 1 & -\frac{5}{3} & -4 \\ 0 & 4 & 1 & 8 \end{array} \right].$$

Now eliminate y from Row3: Row3 $\leftarrow$ Row3 $-4 \times$ Row2:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 1 \\ 0 & 1 & -\frac{5}{3} & -4 \\ 0 & 0 & \frac{17}{3} & 24 \end{array} \right].$$

The third row corresponds to $\frac{17}{3}z = 24$, so $z = \frac{24 \cdot 3}{17} = \frac{72}{17} \approx 4.235$. Substitute z into Row2 equation: $y - \frac{5}{3}z = -4$. So $y = -4 + \frac{5}{3}z = -4 + \frac{5}{3} \cdot \frac{72}{17} = -4 + \frac{360}{51} = -4 + \frac{120}{17} = \frac{-68+120}{17} = \frac{52}{17} \approx 3.059$. (In exact form $y = 52/17$.) Finally substitute y and z into Row1: $x + 2y - z = 1$ so $x = 1 - 2y + z = 1 - 2 \cdot (52/17) + (72/17) = \frac{17}{17} - \frac{104}{17} + \frac{72}{17} = \frac{-15}{17} \approx -0.882$. Thus the solution is $x = -15/17$, $y = 52/17$, $z = 72/17$.

Chapter 3

Intermediate Concepts

3.1 Determinants and Matrix Inverses

The **determinant** is a scalar value associated with a square matrix that encodes certain properties of the linear transformation represented by the matrix. For a 2×2 matrix $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, the determinant is defined as

$$\det(\mathbf{A}) = ad - bc.$$

For larger matrices, the determinant can be computed via a recursive expansion by minors (Laplace expansion) or reduced to the 2×2 case through row operations. The determinant has several important interpretations:

- $\det(\mathbf{A}) = 0$ if and only if the matrix $\mathbf{A}$ is **singular**, meaning it does not have an inverse. In terms of linear transformations, this means $\mathbf{A}$ squashes some non-zero vector to the zero vector, losing information.
- $|\det(\mathbf{A})|$ equals the scaling factor of the volume when the linear transformation $\mathbf{A}$ is applied. For example, if $\det(\mathbf{A}) = 2$ for a 3×3 matrix, it doubles volumes in $\mathbb{R}^3$.
- The sign of $\det(\mathbf{A})$ indicates orientation: a negative determinant means the transformation reverses orientation (like a reflection would).

A closely related concept is the **inverse** of a matrix. A square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is invertible (non-singular) if there exists a matrix $\mathbf{A}^{-1}$ such that:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n,$$

where $\mathbf{I}_n$ is the $n \times n$ identity matrix. If $\det(\mathbf{A}) \neq 0$, the matrix is invertible. For a 2×2 matrix as above, if $\det(\mathbf{A}) = ad - bc \neq 0$, then the inverse is:

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

For larger matrices, computing the inverse can be done via Gaussian elimination (augmenting $\mathbf{A}$ with the identity and reducing) or by using the adjugate (transpose of cofactor matrix) formula, though the former is more practical for computation.

The existence of an inverse is crucial in solving linear systems: if $\mathbf{A}\mathbf{x} = \mathbf{b}$ and $\mathbf{A}^{-1}$ exists, then the unique solution is $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. In machine learning, matrix inverses appear in formulas like the normal equation for linear regression: $\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$, where $(\mathbf{X}^T\mathbf{X})^{-1}$ is the inverse of the Gram matrix of features.

3.2 Transpose of a Matrix

The **transpose** of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is an $n \times m$ matrix denoted $\mathbf{A}^T$, obtained by flipping $\mathbf{A}$ over its diagonal. In other words, the rows of $\mathbf{A}$ become the columns of $\mathbf{A}^T$, and the columns of $\mathbf{A}$ become the rows of $\mathbf{A}^T$. Formally, $(\mathbf{A}^T)_{ij} = \mathbf{A}_{ji}$ for all indices i, j . For example:

$$\mathbf{A} = \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \end{pmatrix}, \quad \mathbf{A}^T = \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 7 & 8 \end{pmatrix}.$$

The transpose is ubiquitous in linear algebra and ML: for instance, the inner product can be written as $\mathbf{u}^T \mathbf{v}$, and in deep learning, it is common to denote by $\mathbf{W}^T$ the transpose of a weight matrix when moving data backward through layers.

A matrix is called **symmetric** if $\mathbf{A}^T = \mathbf{A}$. Symmetric matrices often arise in ML, for example in covariance matrices used in PCA or the Hessian matrix of second derivatives in optimization. Symmetric matrices have special properties like real eigenvalues and orthogonal eigenvectors (by the spectral theorem, which we will encounter later).

3.3 Vector Spaces and Subspaces

Informally, a **vector space** is a collection of vectors that can be added together and scaled by scalars, and still remain within the collection. More formally, a set V is a vector space over a field (like $\mathbb{R}$) if for any $\mathbf{u}, \mathbf{v} \in V$ and any scalars α, β , the linear combination $\alpha \mathbf{u} + \beta \mathbf{v}$ is also in V . Additionally, vector spaces include the existence of a zero vector and additive inverses, and satisfy the usual axioms of associativity, commutativity of addition, etc. The prototypical example is $\mathbb{R}^n$, which is a vector space of dimension n .

A **subspace** is a subset of a vector space that is itself a vector space (using the same addition and scalar multiplication). For example, within $\mathbb{R}^3$, the set of all vectors of the form $(x, y, 0)$ (i.e., those lying in the xy -plane) is a subspace of $\mathbb{R}^3$. To verify a subset $W \subseteq V$ is a subspace, one commonly checks:

- The zero vector of V is in W .
- W is closed under vector addition: if $\mathbf{u}, \mathbf{v} \in W$, then $\mathbf{u} + \mathbf{v} \in W$.
- W is closed under scalar multiplication: if $\mathbf{u} \in W$ and α is any scalar, then $\alpha \mathbf{u} \in W$.

These conditions ensure any linear combination of vectors in W stays in W .

Any vector space has a notion of **span**. Given a set of vectors $S = \{\mathbf{v}_1, \dots, \mathbf{v}_k\} \subseteq V$, the span of S , denoted $\text{span}(S)$, is the set of all linear combinations of those vectors:

$$\text{span}(S) = \{\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_k \mathbf{v}_k \mid \alpha_i \in \mathbb{R}\}.$$

The span of S is always a subspace of V (in fact, the smallest subspace containing all vectors in S). If $\text{span}(S) = V$, we say that S **spans** V or is a spanning set of V .

3.4 Linear Independence, Basis, and Rank

A set of vectors $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ in a vector space V is called **linearly independent** if the only solution to

$$\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_k \mathbf{v}_k = \mathbf{0}$$

is $\alpha_1 = \alpha_2 = \dots = \alpha_k = 0$. In other words, no vector in the set can be written as a linear combination of the others. If a set is not linearly independent, it is **linearly dependent**, meaning one of the vectors can be expressed in terms of the others. For example, in $\mathbb{R}^3$, the vectors $(1, 0, 0)$, $(0, 1, 0)$, $(1, 1, 0)$ are linearly dependent because $(1, 1, 0) = (1, 0, 0) + (0, 1, 0)$.

A **basis** of a vector space V is a set of linearly independent vectors that spans V . Equivalently, a basis is a minimal spanning set or a maximal independent set. Every vector in V can be uniquely expressed as a linear combination of basis vectors. The number of vectors in any basis is the **dimension** of V . For example, $\{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}$ is a basis for $\mathbb{R}^3$ (called the standard basis), and the dimension of $\mathbb{R}^3$ is 3.

The concept of a basis is crucial for understanding coordinate representations and transformations. In ML, feature vectors can be thought of in terms of coordinates relative to a basis of the feature space. Learning algorithms often implicitly assume a basis (like the standard basis of $\mathbb{R}^n$), though techniques like PCA effectively find a new basis (the principal components) that better captures the data's variation.

The **rank** of a matrix $\mathbf{A}$ is the dimension of the subspace spanned by its columns (which is the same as the dimension of the subspace spanned by its rows). This subspace spanned by columns is called the **column space** or **range** of $\mathbf{A}$. Rank tells us how many independent directions the linear transformation $\mathbf{A}$ maps from the domain. If $\mathbf{A}$ is $m \times n$, $\text{rank}(\mathbf{A}) \leq \min(m, n)$. If $\text{rank}(\mathbf{A}) = n$ (full column rank) and $n = m$, then $\mathbf{A}$ is invertible. If $\text{rank}(\mathbf{A}) < n$, the columns are linearly dependent and the system $\mathbf{Ax} = \mathbf{b}$ has either no solutions or infinitely many (depending on $\mathbf{b}$).

In ML, rank is related to the capacity of a linear model to fit data. For instance, if a dataset's features lie in a low-dimensional subspace (say all points lie on a plane in a 3D space), then the feature matrix has low rank. Techniques like matrix factorization or low-rank approximation often leverage the concept of rank to compress data or model complexity.

3.5 Linear Transformations

A **linear transformation** (or linear map) is a function $T : V \rightarrow W$ between two vector spaces that preserves vector addition and scalar multiplication. Formally, for any $\mathbf{u}, \mathbf{v} \in V$ and scalar α , a transformation T is linear if:

$$T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v}),$$

$$T(\alpha\mathbf{u}) = \alpha T(\mathbf{u}).$$

Any linear transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$ can be represented as multiplication by an $m \times n$ matrix (with respect to the standard bases of those spaces). Conversely, any $m \times n$ matrix defines a linear transformation $T(\mathbf{x}) = \mathbf{Ax}$.

Linear transformations include operations like rotations, reflections, scaling, and projections. For example, rotating all points in the plane by 90 degrees about the origin is a linear transformation $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$. If we express T in matrix form relative to the standard basis, that matrix would be $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ (which indeed rotates any vector by 90 degrees counterclockwise). In contrast, a translation (shifting all points by a fixed vector) is not a linear transformation because it does not send the zero vector to zero and fails the additivity property.

Important associated concepts:

- The **kernel** (or null space) of T is the set of vectors $\mathbf{x}$ such that $T(\mathbf{x}) = \mathbf{0}$ in W . For a matrix $\mathbf{A}$, $\ker(\mathbf{A}) = \{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{0}\}$.
- The **image** (or range) of T is the set of vectors in W that are $T(\mathbf{x})$ for some $\mathbf{x} \in V$. For a matrix, the image is the column space mentioned earlier.

The Rank-Nullity Theorem states that for a linear transformation $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$,

$$\dim(\ker(T)) + \dim(\text{Im}(T)) = n.$$

This means the number of independent inputs that map to zero (nullity) plus the number of independent outputs (rank) equals the input dimension.

Understanding linear transformations in ML is essential because many algorithms are either linear transformations or are analyzed using them. For example, each layer in a neural network (without the non-linear activation) is a linear transformation given by the weight matrix. Also, PCA can be seen as a linear transformation that rotates and projects data to a lower-dimensional subspace.

Exercises

1. **Determinant and inverse:** Given $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, compute $\det(\mathbf{A})$. If the matrix is invertible, find $\mathbf{A}^{-1}$. Verify that $\mathbf{A}\mathbf{A}^{-1} = I_2$.

Solution: $\det(\mathbf{A}) = 1 \cdot 4 - 2 \cdot 3 = 4 - 6 = -2$. Since the determinant is not zero, $\mathbf{A}$ is invertible. The inverse for a 2×2 matrix is $\frac{1}{\det(\mathbf{A})}$ times the adjugate:

$$\mathbf{A}^{-1} = \frac{1}{-2} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix}.$$

(Note: 1.5 is $\frac{3}{2}$ and -0.5 is $-\frac{1}{2}$.) We can check $\mathbf{A}\mathbf{A}^{-1} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix} = \begin{pmatrix} (1)(-2) + (2)(1.5) & (1)(1) + (2)(-0.5) \\ (3)(-2) + (4)(1.5) & (3)(1) + (4)(-0.5) \end{pmatrix} = \begin{pmatrix} -2 + 3 & 1 - 1 \\ -6 + 6 & 3 - 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I_2$.

2. **Linear independence:** Determine whether the vectors $\mathbf{v}_1 = (1, 2, 3)$, $\mathbf{v}_2 = (2, 5, 8)$, and $\mathbf{v}_3 = (0, -1, -2)$ in $\mathbb{R}^3$ are linearly independent. If not, find a linear relation between them.

Solution: We set up $\alpha_1(1, 2, 3) + \alpha_2(2, 5, 8) + \alpha_3(0, -1, -2) = (0, 0, 0)$ and solve for scalars α_i . This gives the system of equations:

$$\begin{aligned} \alpha_1 + 2\alpha_2 + 0 \cdot \alpha_3 &= 0, \\ 2\alpha_1 + 5\alpha_2 - 1 \cdot \alpha_3 &= 0, \\ 3\alpha_1 + 8\alpha_2 - 2\alpha_3 &= 0. \end{aligned}$$

From the first equation, $\alpha_1 = -2\alpha_2$. Substitute into the second: $2(-2\alpha_2) + 5\alpha_2 - \alpha_3 = -4\alpha_2 + 5\alpha_2 - \alpha_3 = \alpha_2 - \alpha_3 = 0$, so $\alpha_3 = \alpha_2$. Substitute α_1 and α_3 into the third: $3(-2\alpha_2) + 8\alpha_2 - 2\alpha_3 = -6\alpha_2 + 8\alpha_2 - 2\alpha_2 = 0$, which holds for any α_2 . So

we have a free parameter $\alpha_2 = t$. Let $t = 1$ for simplicity, then $\alpha_2 = 1$, $\alpha_1 = -2$, $\alpha_3 = 1$. This yields the relation:

$$-2\mathbf{v}_1 + 1 \cdot \mathbf{v}_2 + 1 \cdot \mathbf{v}_3 = \mathbf{0},$$

or $\mathbf{v}_2 + \mathbf{v}_3 = 2\mathbf{v}_1$. Since we found a non-trivial combination giving zero, the vectors are linearly dependent. (Indeed, $\mathbf{v}_2 + \mathbf{v}_3$ equals $2\mathbf{v}_1$, showing $\mathbf{v}_1$ is dependent on $\mathbf{v}_2$ and $\mathbf{v}_3$.)

3. **Verifying a subspace:** Let $W = \{(x, y, z) \in \mathbb{R}^3 \mid x + y + z = 0\}$. Show that W is a subspace of $\mathbb{R}^3$ and find a basis for W .

Solution: To show W is a subspace, we check the conditions: - The zero vector $(0, 0, 0)$ satisfies $0 + 0 + 0 = 0$, so $(0, 0, 0) \in W$. - Take two arbitrary vectors in W : (x_1, y_1, z_1) and (x_2, y_2, z_2) with $x_1 + y_1 + z_1 = 0$ and $x_2 + y_2 + z_2 = 0$. Their sum is $(x_1 + x_2, y_1 + y_2, z_1 + z_2)$. Summing components: $(x_1 + x_2) + (y_1 + y_2) + (z_1 + z_2) = (x_1 + y_1 + z_1) + (x_2 + y_2 + z_2) = 0 + 0 = 0$. So the sum is also in W . - For scalar closure, let $\alpha \in \mathbb{R}$ and $(x, y, z) \in W$ with $x + y + z = 0$. Then $\alpha(x, y, z) = (\alpha x, \alpha y, \alpha z)$ and $(\alpha x) + (\alpha y) + (\alpha z) = \alpha(x + y + z) = \alpha \cdot 0 = 0$, so $\alpha(x, y, z) \in W$. Thus W is a subspace.

To find a basis, note the constraint $x + y + z = 0$ implies one variable is determined by the other two, e.g. $z = -x - y$. We can parametrize vectors in W as:

$$(x, y, z) = (x, y, -x - y) = x(1, 0, -1) + y(0, 1, -1).$$

Thus any vector in W is a linear combination of $(1, 0, -1)$ and $(0, 1, -1)$. These two vectors are linearly independent and clearly lie in W . Therefore, $\{(1, 0, -1), (0, 1, -1)\}$ is a basis for W . (There are many possible choices of basis; e.g. $\{(1, 0, -1), (1, 1, -2)\}$ would also work, etc.)

Chapter 4

Advanced Concepts

4.1 Eigenvalues and Eigenvectors

One of the most important concepts in linear algebra is that of eigenvalues and eigenvectors. Given a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, an **eigenvector** is a non-zero vector $\mathbf{v} \in \mathbb{R}^n$ such that $\mathbf{A}\mathbf{v}$ is a scalar multiple of $\mathbf{v}$. That is:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v},$$

for some scalar λ . This scalar λ is called the **eigenvalue** corresponding to eigenvector $\mathbf{v}$. In words, applying the linear transformation $\mathbf{A}$ to $\mathbf{v}$ results in a vector that is in the same (or exactly opposite) direction as $\mathbf{v}$, scaled by λ .

To find eigenvalues of $\mathbf{A}$, we seek nontrivial solutions of $(\mathbf{A} - \lambda I)\mathbf{v} = \mathbf{0}$, which exists if and only if $\det(\mathbf{A} - \lambda I) = 0$. The polynomial $p(\lambda) = \det(\mathbf{A} - \lambda I)$ is called the **characteristic polynomial** of $\mathbf{A}$, and its roots are the eigenvalues. For an $n \times n$ matrix, $p(\lambda)$ is a polynomial of degree n (so there are n eigenvalues in the complex numbers, counting multiplicity). Each eigenvalue λ may have one or more linearly independent eigenvectors. The set of all eigenvectors corresponding to λ , along with the zero vector, forms the **eigenspace** for λ .

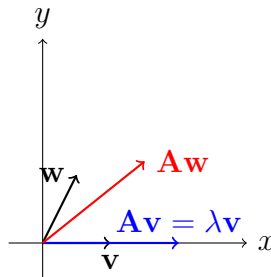


Figure 4.1: Illustration of an eigenvector vs. a general vector under a linear transformation $\mathbf{A}$. $\mathbf{v}$ is an eigenvector since $\mathbf{A}\mathbf{v}$ aligns with $\mathbf{v}$ (scaled by λ). $\mathbf{w}$ is not an eigenvector of $\mathbf{A}$ since $\mathbf{A}\mathbf{w}$ is not parallel to $\mathbf{w}$.

Eigenvalues and eigenvectors play a central role in understanding linear transformations. In the context of ML/DL, eigenvectors can represent "directions of importance". For example, in a covariance matrix of data, the eigenvector associated with the largest eigenvalue indicates the direction of greatest variance (this is the principle behind PCA, to be discussed in the next chapter). In graph analysis or Google's PageRank algorithm,

eigenvectors are used to find steady-state distributions. In deep learning, while eigenvalues/vectors are not directly used in the model architecture, they appear in analyzing the Jacobian or Hessian of the loss function to understand training dynamics (e.g., analyzing saddle points and convergence).

4.2 Eigendecomposition and Diagonalization

If a matrix $\mathbf{A}$ has a full set of linearly independent eigenvectors $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ with corresponding eigenvalues $\{\lambda_1, \dots, \lambda_n\}$, we can form a matrix $\mathbf{P}$ whose columns are these eigenvectors: $\mathbf{P} = [\mathbf{v}_1 \ \mathbf{v}_2 \ \dots \ \mathbf{v}_n]$. Then the matrix $\mathbf{D} = \text{diag}(\lambda_1, \dots, \lambda_n)$ (a diagonal matrix with the eigenvalues on the diagonal) satisfies:

$$\mathbf{AP} = \mathbf{PD}.$$

Multiplying on the right by $\mathbf{P}^{-1}$ (which exists if the eigenvectors are independent and thus $\mathbf{P}$ is invertible), we get:

$$\mathbf{A} = \mathbf{PDP}^{-1}.$$

This is called the **eigendecomposition** of $\mathbf{A}$, and we say $\mathbf{A}$ is **diagonalizable**. In this form, it is easy to compute powers of $\mathbf{A}$: $\mathbf{A}^k = \mathbf{PD}^k\mathbf{P}^{-1}$, since $\mathbf{D}^k$ is just the diagonal matrix of λ_i^k .

Not every matrix is diagonalizable. A matrix may have fewer than n independent eigenvectors (for instance, a defective matrix with a repeated eigenvalue that doesn't yield enough eigenvectors). However, any real symmetric matrix is diagonalizable by an orthogonal matrix $\mathbf{Q}$ (meaning $\mathbf{Q}^{-1} = \mathbf{Q}^T$) such that $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$. This is the spectral theorem and is extremely important in ML because covariance matrices, kernel matrices, and Hessians are often symmetric. Diagonalizing them means finding orthogonal directions (eigenvectors) that decompose the original matrix's action.

In practice, diagonalization helps to decouple linear systems into independent parts. For example, in PCA we diagonalize the covariance matrix of data to find principal components. In differential equations or state-space models, diagonalization simplifies systems of linear differential equations. In deep learning, understanding when a Jacobian matrix can be diagonalized (e.g. via singular values, as in SVD) can give insight into network layers' behaviors.

4.3 LU and QR Decompositions

Besides eigendecomposition, other matrix factorizations are extremely useful for computational purposes:

- **LU Decomposition:** This factors a matrix into a product of a Lower-triangular matrix $\mathbf{L}$ and an Upper-triangular matrix $\mathbf{U}$ (hence "LU"). If $\mathbf{A}$ is an $n \times n$ matrix that can be decomposed (usually requiring that leading principal minors are nonzero or after row pivoting), we get $\mathbf{A} = \mathbf{LU}$. The LU decomposition essentially captures the steps of Gaussian elimination: $\mathbf{L}$ contains the multipliers used to eliminate entries, and $\mathbf{U}$ is the resulting echelon form. LU is useful for efficient solving of linear systems: to solve $\mathbf{Ax} = \mathbf{b}$, solve $\mathbf{Ly} = \mathbf{b}$ (forward substitution) then $\mathbf{Ux} = \mathbf{y}$ (back substitution). This is faster than Gaussian elimination from scratch for multiple $\mathbf{b}$ vectors (multiple right-hand sides).

- **QR Decomposition:** This factors a matrix into the product of an orthonormal matrix $\mathbf{Q}$ and an upper-triangular matrix $\mathbf{R}$. If $\mathbf{A}$ is $m \times n$ with linearly independent columns, $\mathbf{A} = \mathbf{QR}$ where $\mathbf{Q}$ is $m \times n$ with orthonormal columns (so $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$) and $\mathbf{R}$ is $n \times n$ upper triangular. The QR decomposition can be obtained via the Gram-Schmidt process on the columns of $\mathbf{A}$. QR is widely used in numerical linear algebra, for example in solving least squares problems: to solve $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2$, we can write $\mathbf{Ax} = \mathbf{QRx} = \mathbf{Qy}$ (where $\mathbf{y} = \mathbf{Rx}$). Left-multiplying by $\mathbf{Q}^T$ (since $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$) gives $\mathbf{Q}^T \mathbf{Ax} = \mathbf{Q}^T \mathbf{b}$, i.e. $\mathbf{Rx} = \mathbf{Q}^T \mathbf{b}$. Because $\mathbf{R}$ is triangular, this is easy to solve by back-substitution. In ML, QR is useful for implementing linear regression solvers or in eigenvalue algorithms (like the QR algorithm for computing eigenvalues).

4.4 Singular Value Decomposition (SVD)

The **singular value decomposition (SVD)** is one of the most powerful and general matrix factorizations. Any real $m \times n$ matrix $\mathbf{A}$ (not necessarily square) can be decomposed as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where:

- $\mathbf{U}$ is an $m \times m$ orthogonal matrix (its columns are orthonormal eigenvectors of $\mathbf{AA}^T$).
- $\mathbf{V}$ is an $n \times n$ orthogonal matrix (its columns are orthonormal eigenvectors of $\mathbf{A}^T \mathbf{A}$).
- $\mathbf{\Sigma}$ is an $m \times n$ diagonal matrix with non-negative numbers on the diagonal, known as **singular values** of $\mathbf{A}$, typically ordered from largest to smallest. $\Sigma_{ii} = \sigma_i$ are the singular values.

Intuitively, the SVD says any linear transformation $\mathbf{A}$ can be viewed as: rotate/reflect (via $\mathbf{V}^T$), scale along coordinate axes (via $\mathbf{\Sigma}$), and then rotate again (via $\mathbf{U}$). The number of non-zero singular values equals the rank of $\mathbf{A}$.

SVD has numerous applications in ML/DL: - **Low-rank approximation:** By truncating small singular values, we can approximate $\mathbf{A}$ by a lower-rank matrix, which is useful in data compression (e.g., image compression) and noise reduction. The Eckart-Young theorem states that the best rank- k approximation of $\mathbf{A}$ (in least squares sense) is obtained by keeping the top k singular values and corresponding singular vectors. - **Principal Component Analysis (PCA):** As we'll see, performing PCA on a data matrix is effectively computing its SVD and using the singular vectors (or eigenvectors of the covariance) corresponding to the largest singular values (eigenvalues) to project the data into a lower-dimensional subspace. - **Latent Semantic Analysis (LSA) in NLP:** SVD is used on term-document matrices to find latent topics. - Many deep learning optimization methods analyze the singular values of the weight matrices of neural networks to understand issues like vanishing/exploding gradients (where singular values of product of matrices come into play).

Computing SVD is more complex than eigendecomposition but algorithms exist that are efficient and are implemented in numerical libraries. It is a staple tool in the machine learning toolkit due to its broad applicability.

4.5 Orthogonal Projections

An **orthogonal projection** is the operation of projecting a vector onto a subspace such that the error (difference between the vector and its projection) is orthogonal to the subspace. For a given subspace $W \subseteq \mathbb{R}^n$, the projection of a vector $\mathbf{v}$ onto W , denoted $P_W(\mathbf{v})$, is the vector in W closest to $\mathbf{v}$ (in Euclidean distance). If W is spanned by an orthonormal set of vectors $\{\mathbf{u}_1, \dots, \mathbf{u}_k\}$, then:

$$P_W(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{u}_1)\mathbf{u}_1 + \dots + (\mathbf{v} \cdot \mathbf{u}_k)\mathbf{u}_k.$$

In particular, if W is a line spanned by a unit vector $\mathbf{u}$, then $P_W(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{u})\mathbf{u}$, which is the scalar component of $\mathbf{v}$ in direction $\mathbf{u}$ times $\mathbf{u}$.

If the spanning vectors are not orthonormal, one can first orthonormalize them (e.g. via Gram-Schmidt) before using this formula, or use the formula $P = \mathbf{A}(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ for the projection matrix onto the column space of $\mathbf{A}$ (where columns of $\mathbf{A}$ span W). The matrix P is called a **projection matrix** (or projector) and satisfies $P^2 = P$ and $P^T = P$ if it's an orthogonal projection.

Geometrically, orthogonal projection means dropping a perpendicular from $\mathbf{v}$ to the subspace W . The difference $\mathbf{v} - P_W(\mathbf{v})$ is orthogonal to every vector in W .

In machine learning, orthogonal projection is implicit in least squares regression: finding $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}$ that best approximates a target $\mathbf{y}$ in a linear model is projecting $\mathbf{y}$ onto the column space of $\mathbf{X}$. The normal equations $\mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^T \mathbf{y}$ are derived from setting the error orthogonal to the column space. Additionally, techniques like orthogonal matching pursuit in sparse learning, or concept of projection in iterative methods (like projecting gradients in constrained optimization) rely on these principles.

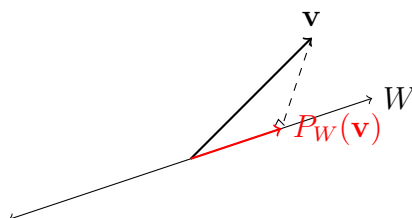


Figure 4.2: Orthogonal projection of $\mathbf{v}$ onto a one-dimensional subspace W (a line through the origin). $P_W(\mathbf{v})$ is the closest point in W to $\mathbf{v}$, and the difference $\mathbf{v} - P_W(\mathbf{v})$ is orthogonal to W .

Exercises

1. **Eigenvalues and diagonalization:** Find the eigenvalues and eigenvectors of $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$. Hence, diagonalize $\mathbf{A}$ if possible.

Solution: The characteristic polynomial is $\det(\mathbf{A} - \lambda \mathbf{I}) = \det \begin{pmatrix} 4 - \lambda & 2 \\ 1 & 3 - \lambda \end{pmatrix} = (4 - \lambda)(3 - \lambda) - 2 \cdot 1 = (4 - \lambda)(3 - \lambda) - 2$. Expanding, $(4 - \lambda)(3 - \lambda) = 12 - 4\lambda - 3\lambda + \lambda^2 = \lambda^2 - 7\lambda + 12$. So $\det(\mathbf{A} - \lambda \mathbf{I}) = \lambda^2 - 7\lambda + 12 - 2 = \lambda^2 - 7\lambda + 10$. Setting this to zero: $\lambda^2 - 7\lambda + 10 = 0$. This factors as $(\lambda - 5)(\lambda - 2) = 0$, giving eigenvalues $\lambda_1 = 5$ and $\lambda_2 = 2$.

For $\lambda_1 = 5$: solve $(\mathbf{A} - 5I)\mathbf{v} = 0$, i.e. $\begin{pmatrix} -1 & 2 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$. The first row $-v_1 + 2v_2 = 0$ implies $v_1 = 2v_2$. Let $v_2 = 1$ (a free parameter), then one eigenvector is $\mathbf{v}^{(1)} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$.

For $\lambda_2 = 2$: solve $(\mathbf{A} - 2I)\mathbf{v} = 0$, i.e. $\begin{pmatrix} 2 & 2 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$. The first row $2v_1 + 2v_2 = 0$ gives $v_1 = -v_2$. Choose $v_2 = 1$, then an eigenvector is $\mathbf{v}^{(2)} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$.

Since we found two linearly independent eigenvectors for the 2×2 matrix, $\mathbf{A}$ is diagonalizable. Let $P = [\mathbf{v}^{(1)} \ \mathbf{v}^{(2)}] = \begin{pmatrix} 2 & -1 \\ 1 & 1 \end{pmatrix}$ and $D = \begin{pmatrix} 5 & 0 \\ 0 & 2 \end{pmatrix}$. We can verify $\mathbf{A}P = PD$. Indeed:

$$\mathbf{A}P = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 2 & -1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} (4*2 + 2*1) & (4*(-1) + 2*1) \\ (1*2 + 3*1) & (1*(-1) + 3*1) \end{pmatrix} = \begin{pmatrix} 10 & -2 \\ 5 & 2 \end{pmatrix},$$

and

$$PD = \begin{pmatrix} 2 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 5 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 2*5 + (-1)*0 & 2*0 + (-1)*2 \\ 1*5 + 1*0 & 1*0 + 1*2 \end{pmatrix} = \begin{pmatrix} 10 & -2 \\ 5 & 2 \end{pmatrix}.$$

Thus $\mathbf{A} = PDP^{-1}$, with P^{-1} easily computable or implied by the verification above.

2. **Projection onto a line:** Let W be the line in $\mathbb{R}^2$ spanned by $\mathbf{w} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$. Compute the orthogonal projection of $\mathbf{v} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$ onto W .

Solution: First, a unit vector in the direction of $\mathbf{w}$ is $\mathbf{u} = \frac{1}{\sqrt{3^2+1^2}} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ 1 \end{pmatrix}$. The projection of $\mathbf{v}$ onto W is:

$$P_W(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{u}) \mathbf{u} = \left(\begin{pmatrix} 2 & 2 \end{pmatrix} \cdot \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ 1 \end{pmatrix} \right) \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

Compute the dot product: $\mathbf{v} \cdot \mathbf{u} = \frac{1}{\sqrt{10}}(2*3 + 2*1) = \frac{1}{\sqrt{10}}(6 + 2) = \frac{8}{\sqrt{10}}$. So,

$$P_W(\mathbf{v}) = \frac{8}{\sqrt{10}} \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \frac{8}{10} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{24}{10} \\ \frac{8}{10} \end{pmatrix} = \begin{pmatrix} 2.4 \\ 0.8 \end{pmatrix}.$$

Thus, $P_W(\mathbf{v}) = \begin{pmatrix} 2.4 \\ 0.8 \end{pmatrix}$. We can double-check that $\mathbf{v} - P_W(\mathbf{v}) = (2 - 2.4, 2 - 0.8) = (-0.4, 1.2)$ is orthogonal to $\mathbf{w}$ by checking the dot product: $(-0.4, 1.2) \cdot (3, 1) = -1.2 + 1.2 = 0$, as expected.

Chapter 5

Advanced Topics for ML/DL

5.1 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a technique for reducing the dimensionality of data while preserving as much variance as possible. It is fundamentally a linear algebra method: given a dataset, PCA finds a new orthogonal basis for the data space such that the first axis (principal component) captures the largest variance in the data, the second axis captures the next largest variance orthogonal to the first, and so on.

Mathematically, suppose we have a dataset of m samples in n dimensions, represented as an $m \times n$ data matrix $\mathbf{X}$ (each row is a sample, each column a feature). Typically, we first center the data (subtract the mean of each column) so that each feature has mean 0. PCA can be derived by seeking the direction $\mathbf{w} \in \mathbb{R}^n$ (unit vector) that maximizes the variance of the data when projected onto that direction:

$$\text{maximize } \text{Var}(\mathbf{X}\mathbf{w}) = \frac{1}{m} \|\mathbf{X}\mathbf{w}\|^2 = \frac{1}{m} \mathbf{w}^T (\mathbf{X}^T \mathbf{X}) \mathbf{w},$$

subject to $\|\mathbf{w}\| = 1$. Here $\mathbf{X}^T \mathbf{X}$ (an $n \times n$ matrix) is proportional to the covariance matrix of the features. This is a constrained optimization that can be solved using Lagrange multipliers, leading to:

$$(\mathbf{X}^T \mathbf{X}) \mathbf{w} = \lambda \mathbf{w},$$

which is an eigenvalue problem. λ here plays the role of the variance captured by the projection $\mathbf{w}$. The equation says that the optimal $\mathbf{w}$ is an eigenvector of $\mathbf{X}^T \mathbf{X}$. In fact, the $\mathbf{w}$ that maximizes variance is the principal eigenvector (the one with largest eigenvalue). The value λ is the corresponding eigenvalue, which equals m times the variance along $\mathbf{w}$.

Thus, the first principal component is the eigenvector of the covariance matrix with the largest eigenvalue. Subsequent principal components are given by the next eigenvectors (with the next largest eigenvalues), subject to being orthogonal to previous ones. If we take k principal components (eigenvectors), we can form a matrix $\mathbf{W}_{n \times k}$ whose columns are those eigenvectors. The transformed data (of reduced dimension k) is then $\mathbf{X}_{m \times n} \mathbf{W}_{n \times k} = \mathbf{X}_{m \times n}$ projected onto the subspace spanned by those k eigenvectors.

The result of PCA is a set of new features (principal components) that are uncorrelated and ordered by the amount of variance they explain. In practice, PCA is often computed via the SVD: if $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^T$ (with $m > n$ assume), then columns of $\mathbf{V}$ are eigenvectors of $\mathbf{X}^T \mathbf{X}$, and singular values in Σ relate to the square roots of the eigenvalues. The top k singular vectors (columns of $\mathbf{V}$) correspond to the top k principal components.

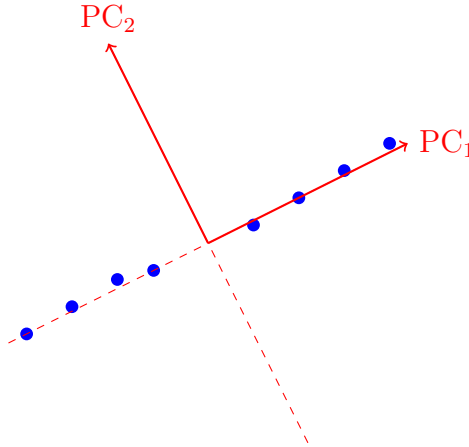


Figure 5.1: A dataset of points (blue) in 2D and its first two principal components (red axes). PC_1 captures the largest variance, and PC_2 is orthogonal to PC_1 capturing the next variance. Projecting points onto PC_1 (the horizontal red axis) would give a one-dimensional representation preserving maximal variance.

PCA is widely used in ML for:

- **Dimensionality reduction:** reducing computational complexity and mitigating the curse of dimensionality.
- **Noise reduction:** by dropping components with very low variance, which often represent noise.
- **Visualization:** high-dimensional data can be projected to 2D or 3D for plotting.
- It also underpins more complex techniques like kernel PCA (a non-linear extension).

5.2 SVD and Data Compression

Singular Value Decomposition (introduced in the previous chapter) has direct applications in data compression and latent factor models. Since SVD can approximate a matrix with fewer components (rank reduction), one common use is image compression. For example, an image can be represented as a matrix (for grayscale, entries are pixel intensities). By computing its SVD and keeping only the largest k singular values (and corresponding singular vectors), one can store far less data than the original image while retaining the main structure. The compressed image is $\mathbf{U}_k \Sigma_k \mathbf{V}_k^T$ where Σ_k has only the top k singular values (and zeros elsewhere), and $\mathbf{U}_k$, $\mathbf{V}_k$ have the corresponding singular vectors. As k increases, the approximation improves, and at $k = \text{rank}(\mathbf{A})$ it's exact.

In Natural Language Processing (NLP), SVD is used in Latent Semantic Analysis (LSA) on term-document matrices. The matrix decomposition reveals latent "concepts" or topics; truncating the SVD can reduce noise and improve the detection of synonymy by focusing on dominant relationships.

Another example: In recommender systems, a user-item ratings matrix can be decomposed by SVD or related techniques (like matrix factorization via gradient descent) to find latent factors that explain user preferences and item properties, enabling recommendations by reconstructing an approximation of the matrix.

Thus, SVD provides a principled way to compress information by discarding directions (singular vectors) associated with small singular values, which often correspond to minor details or noise.

5.3 Eigenvalues in Dimensionality Reduction

Eigenvalues guide us in deciding how many dimensions to keep in a reduced representation. In PCA, after computing the eigenvalues of the covariance matrix (or singular values of the data matrix), one can look at the magnitude of these eigenvalues to determine the importance of each principal component. A common heuristic is to retain enough principal components to explain a certain percentage (say 95%) of the variance. If $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ are the eigenvalues of the data covariance, and we choose k such that:

$$\frac{\lambda_1 + \lambda_2 + \dots + \lambda_k}{\lambda_1 + \lambda_2 + \dots + \lambda_n} \approx 0.95,$$

then those k components explain about 95% of the variance in the data.

Plotting the eigenvalues (or singular values) in order often produces a "scree plot" where one looks for an "elbow" – a point where adding more components yields diminishing returns. The eigenvalues essentially tell us the intrinsic dimensionality of the data (how many significant directions there are).

In deep learning, while PCA is not typically performed on the raw data (because we often use nonlinear methods to reduce dimension), analysis of covariance of features or activations can still reveal how information is compressed in layers. Also, the concept of eigenvalues is relevant in understanding the Fisher information matrix or Hessian of the model: a large spread in eigenvalues (called high condition number) means some directions in parameter space are much more sensitive, which can make optimization difficult.

5.4 Optimization: Gradient Descent and the Hessian

Most machine learning models are trained by optimizing a cost function (loss function). Gradient descent is a fundamental algorithm to perform this optimization. It relies heavily on linear algebra: - The model parameters can be thought of as a vector $\mathbf{w}$ in $\mathbb{R}^d$. - The gradient of the loss function $L(\mathbf{w})$ with respect to $\mathbf{w}$ is a vector in $\mathbb{R}^d$ that points in the direction of steepest increase of L (and its negative is the direction of steepest decrease).

Gradient descent iteratively updates the parameter vector:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha \nabla L(\mathbf{w}_{\text{old}}),$$

where α is the learning rate (a small positive scalar) and $\nabla L(\mathbf{w})$ is the gradient vector. This is essentially a linear algebra operation: subtracting a scalar multiple of one vector from another. In practice, for a model like linear regression $L(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, the gradient is $\nabla L = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ (which comes from linear algebra derivation using $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{y}$). For deep networks, $\nabla L(\mathbf{w})$ is computed via backpropagation, which itself is a sequence of linear algebra operations (multiplying gradients by weight matrices, etc., in reverse).

The **Hessian** matrix H of second derivatives (an $d \times d$ matrix where $H_{ij} = \frac{\partial^2 L}{\partial w_i \partial w_j}$) plays a key role in advanced optimization methods. Newton's method for optimization uses the Hessian to take curvature into account:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - H^{-1} \nabla L(\mathbf{w}_{\text{old}}),$$

assuming H is invertible. While Newton's method can converge faster (it is akin to a quadratic approximation of the function), computing and inverting the Hessian is often

impractical for large d (which in deep learning can be millions of parameters). However, understanding the Hessian is useful: - The eigenvalues of the Hessian indicate the curvature along eigen-directions. If all eigenvalues are positive, we are at a local minimum; if some are negative, it's a saddle point or local maximum. - Ill-conditioning (a very large ratio between the largest and smallest eigenvalue of H) means gradient descent might zigzag and converge slowly, because the surface is steep in some directions and flat in others. Techniques like preconditioning or adaptive gradient methods (AdaGrad, Adam) can be seen as attempts to normalize or rescale the problem so that all directions are treated appropriately. - In second-order methods or in analyzing convergence, one might diagonalize or approximate the Hessian to understand the dynamics. For example, if $H = Q\Lambda Q^T$ (eigendecomposition with Λ diagonal of eigenvalues), in the eigen-basis, gradient descent on a quadratic function $q(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T H \mathbf{w}$ decouples into independent scalar problems for each eigenvalue λ_i : $w_i \leftarrow w_i - \alpha \lambda_i w_i$. This update converges if $|1 - \alpha \lambda_i| < 1$ for all i , which gives a condition on α relative to the eigenvalues.

In summary, linear algebra is deeply ingrained in how we optimize ML models: from expressing gradients and Hessians, to diagnosing optimization issues via eigenvalues, to even designing optimization algorithms that operate in the space of these linear algebraic structures.

Exercises

1. **PCA variance explained:** Suppose you have a dataset in $\mathbb{R}^5$ (5-dimensional) and you perform PCA on the covariance matrix. The eigenvalues (variances along principal components) are found to be $\{4.2, 1.3, 0.5, 0.1, 0.0\}$.
 - (a) What is the percentage of variance explained by the first principal component?
 - (b) How many dimensions would you choose to keep if you desire to retain at least 95% of the variance?

Solution:

- (a) The total variance is $4.2 + 1.3 + 0.5 + 0.1 + 0.0 = 6.1$. The first component has variance 4.2. The percentage explained by the first component is $\frac{4.2}{6.1} \approx 0.6885$, which is about 68.85%.
- (b) To retain at least 95% of the variance, we sum the top eigenvalues until we reach 95

2. **Gradient descent step:** Consider a simple quadratic function $L(w) = \frac{1}{2}aw^2$ for some constant $a > 0$.

- (a) Write the update rule for gradient descent on this function (for parameter w) with learning rate α .
- (b) If $a = 50$, what is the condition on α for gradient descent to converge?

Solution:

- (a) We have $\frac{dL}{dw} = aw$. Gradient descent update: $w_{\text{new}} = w_{\text{old}} - \alpha(aw_{\text{old}}) = (1 - \alpha a)w_{\text{old}}$.

- (b) Gradient descent will converge (in this simple one-dimensional case) if $|1 - \alpha a| < 1$. Plugging $a = 50$: $|1 - 50\alpha| < 1$. This inequality holds when $-1 < 1 - 50\alpha < 1$. The right side gives $1 - 50\alpha < 1 \implies -50\alpha < 0 \implies \alpha > 0$ (which we assume anyway). The left side gives $-1 < 1 - 50\alpha \implies -2 < -50\alpha \implies \alpha < \frac{2}{50} = 0.04$. So $0 < \alpha < 0.04$ is the condition for convergence.

Bibliography

- [1] Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.